

Activitat: La granota i l'escorpí

MÒDUL M03		



[La falla de la granota i l'escorpí](#)

Índex

Índex	1
1-. Sobre classes i mètodes abstractes, interfícies, pèrdua d'identitat i polimorfisme	1
Oracle tutorial	2
https://docs.oracle.com/javase/tutorial/java/concepts/index.html	2
Exemple a classe	2
Figura	2
Rectangle	3
Cercle	4
Main	5
FiguraInterface	6
Interfícies	6
Polimorfisme	7
Pèrdua d'identitat	7

1-. Sobre classes i mètodes abstractes, interfícies, pèrdua d'identitat i polimorfisme

Oracle tutorial

<https://docs.oracle.com/javase/tutorial/java/concepts/index.html>

Exemple a classe

Figura

Cercle, Triangle i Rectangle

1. Té sentit que hi hagi figures que no siguin Cercle, Triangle o Rectangle?
2. Volem que en totes les figures es pugui calcular l'àrea i el perímetre.

Figura

```
import java.awt.*;

public abstract class Figura implements FiguraInterface {
    private Point point;
    private Color color;

    public Figura(Point point, Color color) {
        this.point = point;
        this.color = color;
    }

    public Point getPoint() {
        return point;
    }

    public void setPoint(Point point) {
        this.point = point;
    }

    public Color getColor() {
        return color;
    }

    public void setColor(Color color) {
        this.color = color;
    }

    @Override
    public String toString() {
```

```
        return "cat.iticbcn.pguitart.Figura{" +  
            "point=" + point +  
            ", color=" + color +  
            "'}';  
    }  
  
    public void quiSoc(){  
        System.out.println("soc una Figura");  
    }  
}
```

Rectangle

```
import java.awt.*;  
  
public class Rectangle extends Figura {  
    private int base;  
    private int alçada;  
  
    public Rectangle(Point point, Color color, int base, int alçada) {  
        super(point, color);  
        this.base = base;  
        this.alçada = alçada;  
    }  
  
    public int getBase() {  
        return base;  
    }  
  
    public void setBase(int base) {  
        this.base = base;  
    }  
  
    public int getAlçada() {  
        return alçada;  
    }  
  
    public void setAlçada(int alçada) {  
        this.alçada = alçada;  
    }  
  
    @Override  
    public void quiSoc(){  
        System.out.println("soc un rectangle");  
    }  
  
    @Override  
    public String toString() {  
        return "Rectangle{" +
```

```
        "base=" + base +  
        ", alçada=" + alçada +  
        '}' + super.toString();  
    }  
  
    @Override  
    public double area() {  
        return 0;  
    }  
  
    @Override  
    public double perimetre() {  
        return 0;  
    }  
}
```

Cercle

```
import java.awt.*;  
  
public class Cercle extends Figura {  
    private int radi;  
  
    public Cercle(Point point, Color color, int radi) {  
        super(point, color);  
        this.radi = radi;  
    }  
  
    public int getRadi() {  
        return radi;  
    }  
  
    public void setRadi(int radi) {  
        this.radi = radi;  
    }  
  
    @Override  
    public String toString() {  
        return "Cercle{" +  
            "radi=" + radi +  
            '}' + super.toString();  
    }  
  
    @Override  
    public void quiSoc(){  
        System.out.println("soc un cercle");  
    }  
  
    @Override
```

```
public double area() {  
    return 0;  
}  
  
@Override  
public double perimetre() {  
    return 0;  
}  
}
```

Main

```
import java.awt.*;  
  
public class Main {  
    public static void main(String[] args) {  
  
        //si figura és abstracta no puc crear cap Figura  
        //Figura figura = new Figura(new Point(100, 100), Color.RED);  
        Rectangle rectangle = new Rectangle(new Point(), new  
Color(0,255,0), 50, 30);  
        //System.out.println(figura);  
        System.out.println(rectangle);  
  
        Figura figura = new Rectangle(new Point(50,50), Color.BLUE,  
40, 50);  
        Object objecte = new Rectangle(new Point(50,50), Color.BLUE,  
40, 50);  
        //Figura és un rectangle, que no sap que ho és. Ha perdut  
identitat  
        //figura. no mostra els mètodes de Rectangle  
  
        Cercle cercle = new Cercle(new Point(), Color.YELLOW, 50);  
  
        //r és un Rectangle?  
        System.out.println("r és un Rectangle? " + (rectangle  
instanceof Rectangle));  
        System.out.println("r és una Figura? " + (rectangle instanceof  
Figura));  
        System.out.println("r és un Object? " + (rectangle instanceof  
Object));  
        System.out.println("r és un FiguraInterface? " + (rectangle  
instanceof FiguraInterface));  
        //f és un Rectangle?  
        System.out.println("figura és un Rectangle? " + (figura  
instanceof Rectangle));  
        System.out.println("figura és una Figura? " + (figura  
instanceof Figura));  
    }  
}
```

```
        System.out.println("figura és un Object? " + (figura
instanceof Object));
        //object és un Rectangle?
        System.out.println("object és un Rectangle? " + (objecte
instanceof Rectangle));
        System.out.println("object és una Figura? " + (objecte
instanceof Figura));
        System.out.println("object és un Object? " + (objecte
instanceof Object));
        //QUI SOM
        rectangle.quiSoc(); //un rectangle
        figura.quiSoc(); //un rectangle (es comporta com el que és)
        ((Rectangle) (objecte)).quiSoc(); //un rectangle
        cercle.quiSoc(); //un cercle

        //Polimorfisme
        Figura figura1 = new Cercle(new Point(10, 10), Color.RED, 10);
        Figura figura2 = new Rectangle(new Point(0,0), Color.BLUE, 10,
10);
        figura1.quiSoc();
        figura2.quiSoc();
    }
}
```

FiguraInterface

```
public interface FiguraInterface {

    public double area();
    public double perimetre();
}
```

Interfícies

Una interfície és una classe abstracta en la qual tots els mètodes són abstractes. (Només té les capçaleres dels mètodes, però no els mètodes). Com que tots els mètodes són abstractes no cal posar el descriptor **abstract**.

Les subclasses d'una interfície han d'implementar tots els mètodes de la interfície. D'aquí que es declari amb la paraula clau **implements**.

```
interface B {  
    public double area();  
    public double superficie();  
}  
  
interface C {  
    public double area();  
}  
  
class A implements B, C {  
    @Override  
    public double area(){  
        ...  
    }  
  
    @Override  
    public double superficie(){  
        ...  
    }  
}
```

Noteu que, a diferència de l'herència múltiple, quan una instància de A executa àrea no hi ha confusió per saber si executa la de B o la de C, ja que executarà la seva pròpia implementació.

Les interfícies es poden veure com un **contracte** que signa la classe que l'implementa en el qual es compromet a implementar els mètodes abstractes seguint les indicacions que es donen en la declaració de la interfície.

Polimorfisme

A l'exemple hem comentat que, encara que la classe Figura sigui abstracta, és possible assignar un triangle o un rectangle, a una variable de classe Figura:

```
Figura triangle = new Triangle(); //ES CORRECTE ja que un Triangle és una Figura.  
Figura rectangle = new Rectangle(); //ES CORRECTE
```

Pèrdua d'identitat

Es diu que triangle o rectangle han **perdut identitat** ja que es pensen que és una Figura (no saben que són un triangle o un rectangle).

Aquest fet és la clau del **polimorfisme**, una de les propietats més interessants de la programació orientada a objectes. Fixem-nos que tan *triangle* com *rectangle* són, segons la declaració, de tipus Figura. En canvi:

- Si cridem `triangle.area()` es calcularà l'àrea com un triangle (ja que internament hem creat `triangle` com un `triangle`).
- Si cridem `rectangle.area()` es calcularà l'àrea com un rectangle (ja que internament hem creat `rectangle` com un `rectangle`).

Per tant, tenim dos objectes que tot i ser del mateix tipus es comporten de manera diferent (calculen l'àrea de forma diferent dependent del que són realment.)

Recordeu que l'escorpí de la falla, per molt que ho dissimuli, és un escorpí i, per tant, acaba comportant-se com el que és. Pica la granota encara que amb això hi deixi la pell.

Aquesta propietat ofereix possibilitats molt grans a la programació orientada a objectes, que ja anirem veient més endavant. La idea, però, és que permet un nivell encara més alt de reutilització de codi: per exemple permet reutilitzar algorismes com ara l'algorisme d'ordenació d'arrays amb `Arrays.sort()`.

Exemple de classe